# Angewandte Bioinformatik

**24. Juni 2026, SS 2026**

## Aufgabe 8.1

### a)

Siehe `onehot.py`, das Skript ist an dem gleichnamigen Skript aus der Vorlesung orientiert, der Aufruf geschieht analog. Die erzeugten Dateien sind im Ordner `Data`

### b)

Es wird ein neuronales Netz vom Typ CNN erstellt, da es seine Funktionalität um Eigenschaften bzw. Muster auf Sequenzen zu erkennen (z. B. bei DeepBind), bewährt hat. Es werden typische Schichten für CNNs, neben Convolutional Layers mit nichtlinearen Aktivierungsfkt. auch Pooling Layers und voll verbundene Schichten genutzt (Fully-Connected-Layers).

Es wird als Fehlerfunktion der Mean Absolute Error genutzt, da die Zielwerte zwischen 0 und 1 liegen und sich daher der Mean-Squared-Error nicht gut eignet.

Es werden bestimmte Hyperparameter variiert, um eine bessere Performanz auf der Regressionsaufgabe zu erzielen. Dabei wird eine 5-Fache Cross-Validation auf den Trainingsdaten durchgeführt. Es wurde kein fester Validierungsdatensatz erstellt, da der kleine „MAX"-Transkriptionsfaktor-Datensatz nicht noch weiter beschränkt werden soll. Außerdem werden dadurch mehrere Initialisierungen des Modells getestet. Die Sequenzen mit Länge 12 werden durch symmetrisches Padding (mit 0) auf die Länge der Sequenzen mit 14 Nukleotiden gebracht.

Die Ergebnisse der kleinen Hyperparametersuche wurden vom Kreuzvalidierungs-Skript `hyperparam_search.py` in die Datei `hyp_search.py` geschrieben. Es werden zunächst 6 verschiedene Architekturen mit sonst gleichen Settings getestet. Dabei können wir beobachten, dass ein komplexeres Modell zu besseren Ergebnissen führt. Bei jeder Architektur ist ein gut funktionierendes Training zu beobachten (siehe Auswahl im Ordner Plots). Da die Sequenzen in der One-Hot-Codierung pro Base 4 Werte enthalten, werden auch die Kernel-Größen und Filter-Anzahlen als Vielfache von 4 gewählt.

Es wurden auch weitere Hyperparameter getestet, allerdings führten diese nicht zu besseren Ergebnissen, wie dies in der `tsv`-Datei zusehen ist.

### c)

Das Skript `train_predict.py` trainiert das Modell mit den besten Hyperparametern, berechnet außerdem die Vorhersagen für die Testdaten der 3 Transkriptionsfaktoren und speichert diese im gewünschten Format ab.

## d)

Das Skript `evaluate.py` evaluiert die Güte der Vorhersagen, des in der vorherigen Teilaufgabe trainierten Modells mittels Pearson-Korrelation. Diese wird zwischen den tatsächlichen TF-Bindewahrscheinlichkeiten und den vorhergesagten berechnet.

Das Programm gibt folgende Korrelationen aus:
Pearson correlation for HSF1: 0.3526
Pearson correlation for MAX: 0.2081
Pearson correlation for VSX1: 0.4904

Die Korrelationen sind nicht sehr hoch, was zeigt, dass die Vorhersagen ziemlich schlecht sind. Immerhin diese positiv. Bei VSX1 ist die Korrelation deutlich am höchsten, obwohl es für HSF1 das 1,5-fache der Trainingsdaten gibt von VSX1 gibt, ist das Modell bei der Vorhersage von HSF deutlich schlechter durch die niedrigere Korellation. Möglicherweise liegt hier ein Overfitting vor, da die Trainingsfehler (MAE) bei Epoche 10 bei HSF1: 0,0673 fast so groß wie bei VSX1: 0,0657 ist. Möglicherweise tritt auch bei VSX1 Overfitting auf.

Bei MAX ist die Korrelation noch einmal stark geringer als bei HSF1, dies könnte an der geringen Zahl an Trainingsdaten liegen.